

AI VISUAL INSPECTION

DefectScope

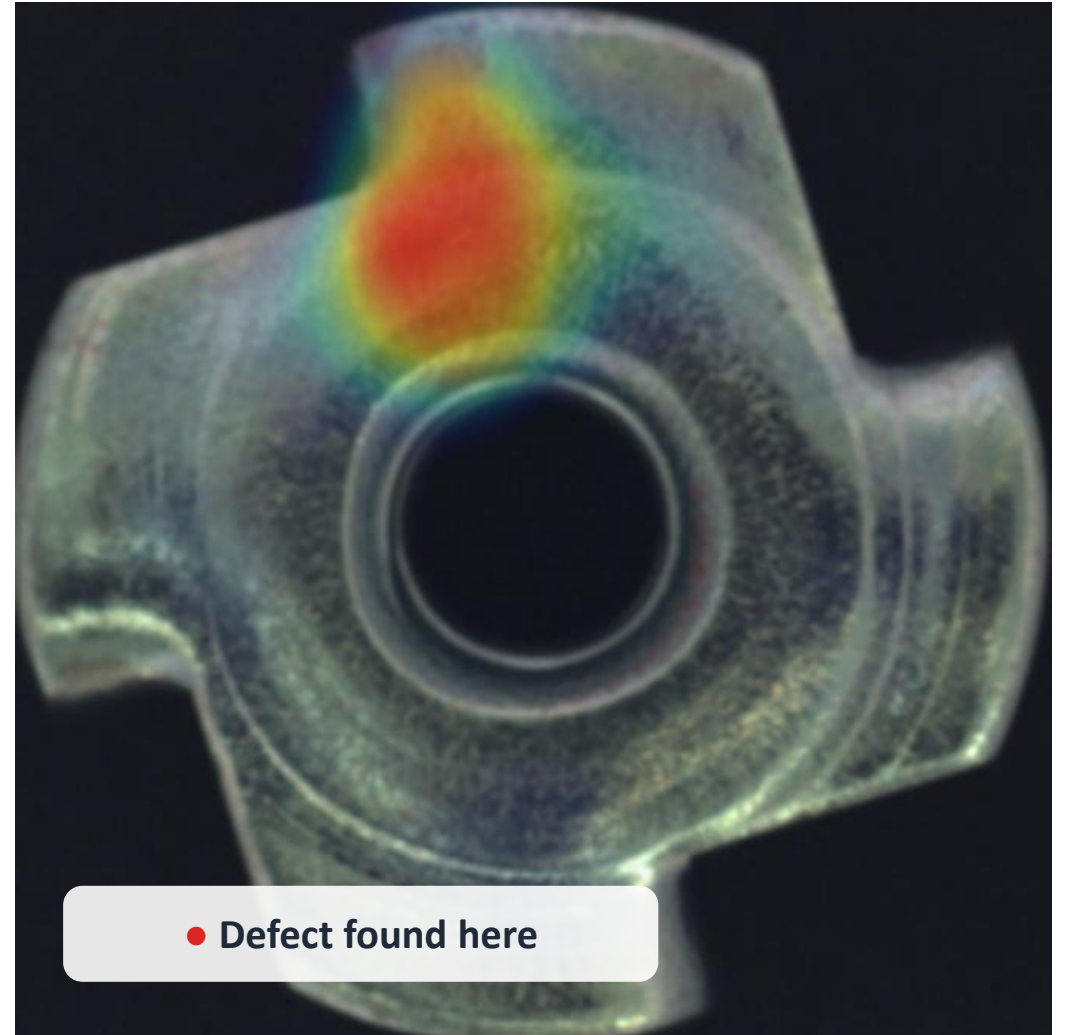
Spots scratches and dents.

No defect photos needed.

No defect labels

Pixel heatmap

Runs on CPU



THE PROBLEM

Rare defects make AI inspection hard.

1

Normal AI needs defect photos

A standard AI model must see hundreds of examples of every defect type. A good production line simply doesn't have them.

2

New defects slip through

A scratch, crack or contamination nobody planned for goes straight past a model that was never shown it.

3

Manual checks don't scale

People get tired, disagree with each other, and can't check every part at line speed.

What you do have: photos of good parts. **That's all DefectScope needs.**

TRY IT YOURSELF

Upload a photo. See pass/fail and where.

AI VISUAL INSPECTION

DefectScope

Spots scratches and dents. **No defect photos needed.**

No defect labels Pixel heatmap Runs in your browser

1 - Product line

Metal nut

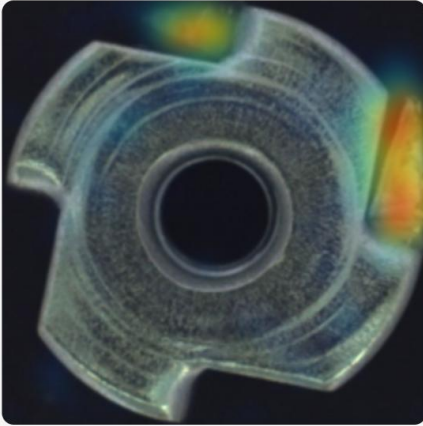
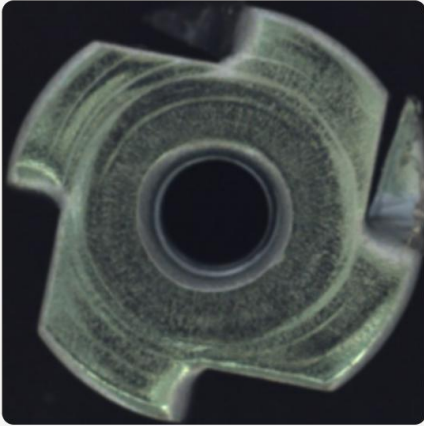
2 - Pick a sample



bent color good

...or upload your own photo

Upload a photo of the selected product type: the model only knows that line's normal look.
🔒 Your photo never leaves your device; the AI runs in this page.



Input Where it looks wrong

⚠️ **DEFECT**

Anomaly score 1.29x the pass/fail threshold (above 1.00x = defect) 211 ms in your browser

1

Set up from good parts only

Each product line was set up from 209–320 photos of good parts. No defect photos, no labelling.

2

Shows the operator where

A heatmap marks the exact spot, so people can check it in seconds.

3

A pass/fail line you can trust

The reject threshold is set on good parts the model never saw, not tuned on the test set.

RESULTS IN PLAIN NUMBERS

What it caught, what it wrongly rejected, how fast.

94%

of defective parts caught
average over 5 product types

7%

of good parts wrongly rejected
the false alarms that cost you money

50 ms

per part on a normal CPU (≈1,200
parts/min)
2 CPU threads, no GPU

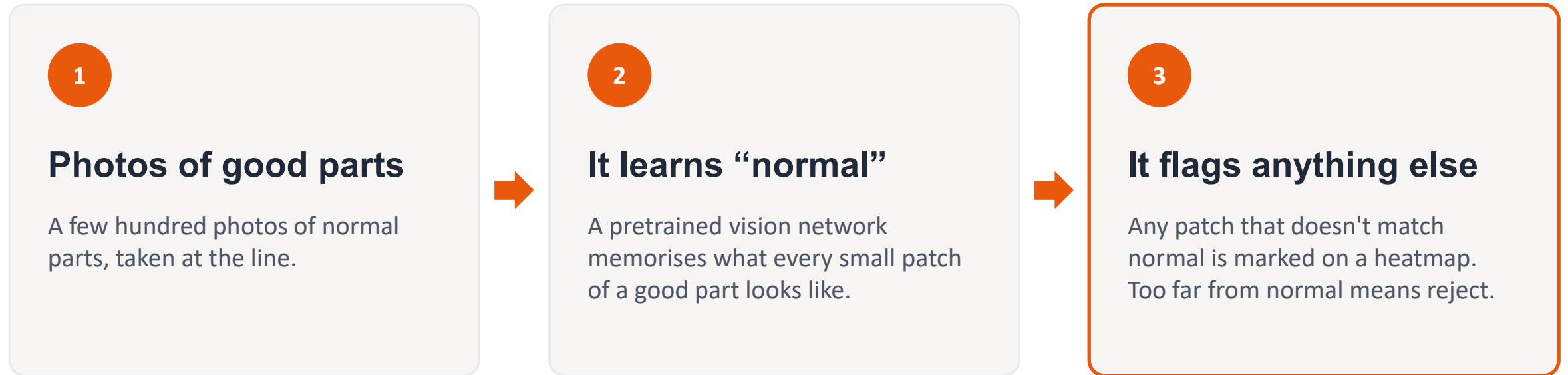
Product	Defective parts caught	Good parts wrongly rejected
Bottle	98% (62 of 63)	0% (0 of 20)
Screw	76% (90 of 119)	0% (0 of 41)
Carpet	99% (88 of 89)	25% (7 of 28)
Metal nut	98% (91 of 93)	5% (1 of 22)
Transistor	100% (40 of 40)	3% (2 of 60)

How this was measured

Public MVTec AD test images the model never saw, with one fixed pass/fail threshold per product. On your parts, the feasibility step measures these same numbers before you commit.

HOW IT WORKS

Three steps. No labelling project.

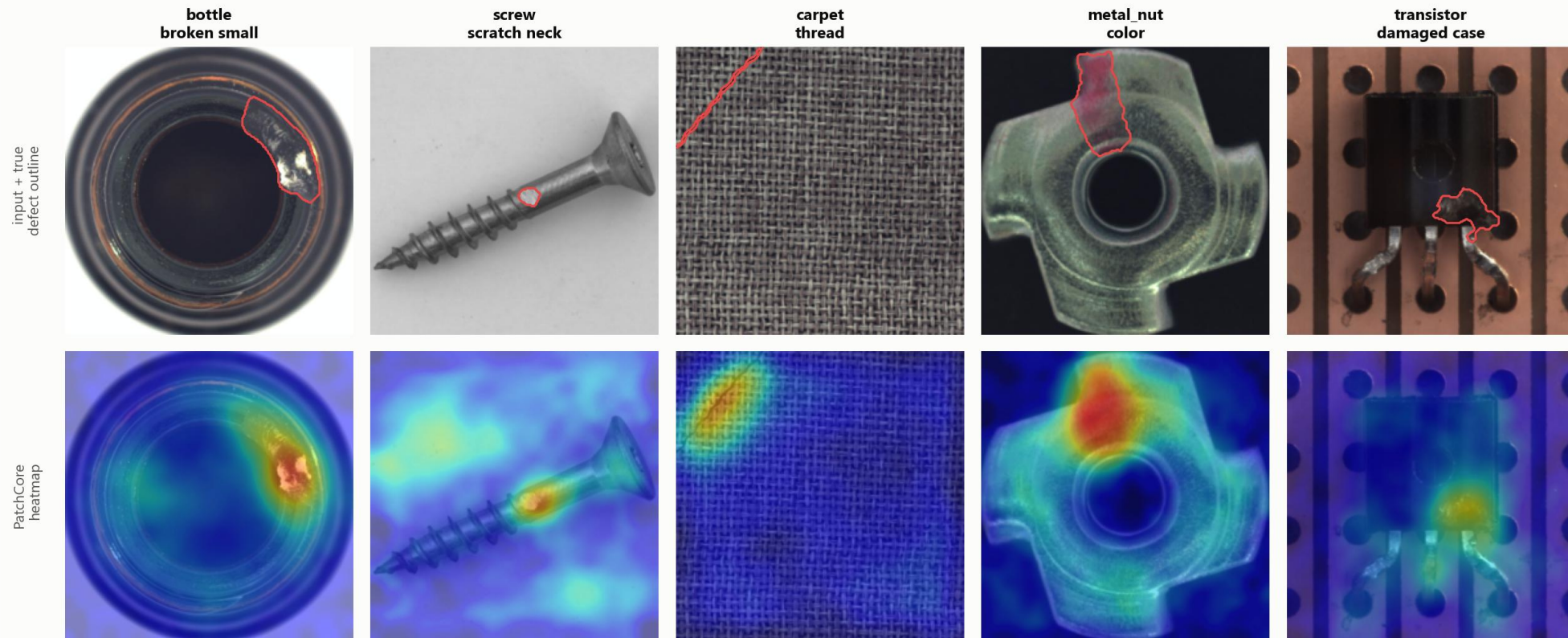


For engineers: a from-scratch re-implementation of PatchCore (Roth et al., CVPR 2022) in PyTorch: mid-level features from a frozen backbone, a greedy-coreset memory bank, and nearest-neighbour patch scoring. Benchmarks are in the appendix.

REAL DETECTIONS

Defects found on parts it never saw.

Per product: the defective test photo with the median score. Red outline = where the real defect is.



HOW WE'D WORK TOGETHER

Start small. See it work on your parts first.

STEP 1

Send photos

A few hundred photos of good parts from your line (this demo used 209–320), plus any defective ones you have.

STEP 2

Feasibility report

I set it up on your parts and report the real numbers: defects caught, false rejects, speed.

STEP 3

Deploy at the line

Runs on a PC or edge box next to your camera, with a pass/fail signal your line can act on.

What you get: the trained inspector for your product · an operator app or API · source code you own · setup docs so your team can add new products themselves.

NEXT STEP

Send me photos of your parts.
I'll show you if it works.



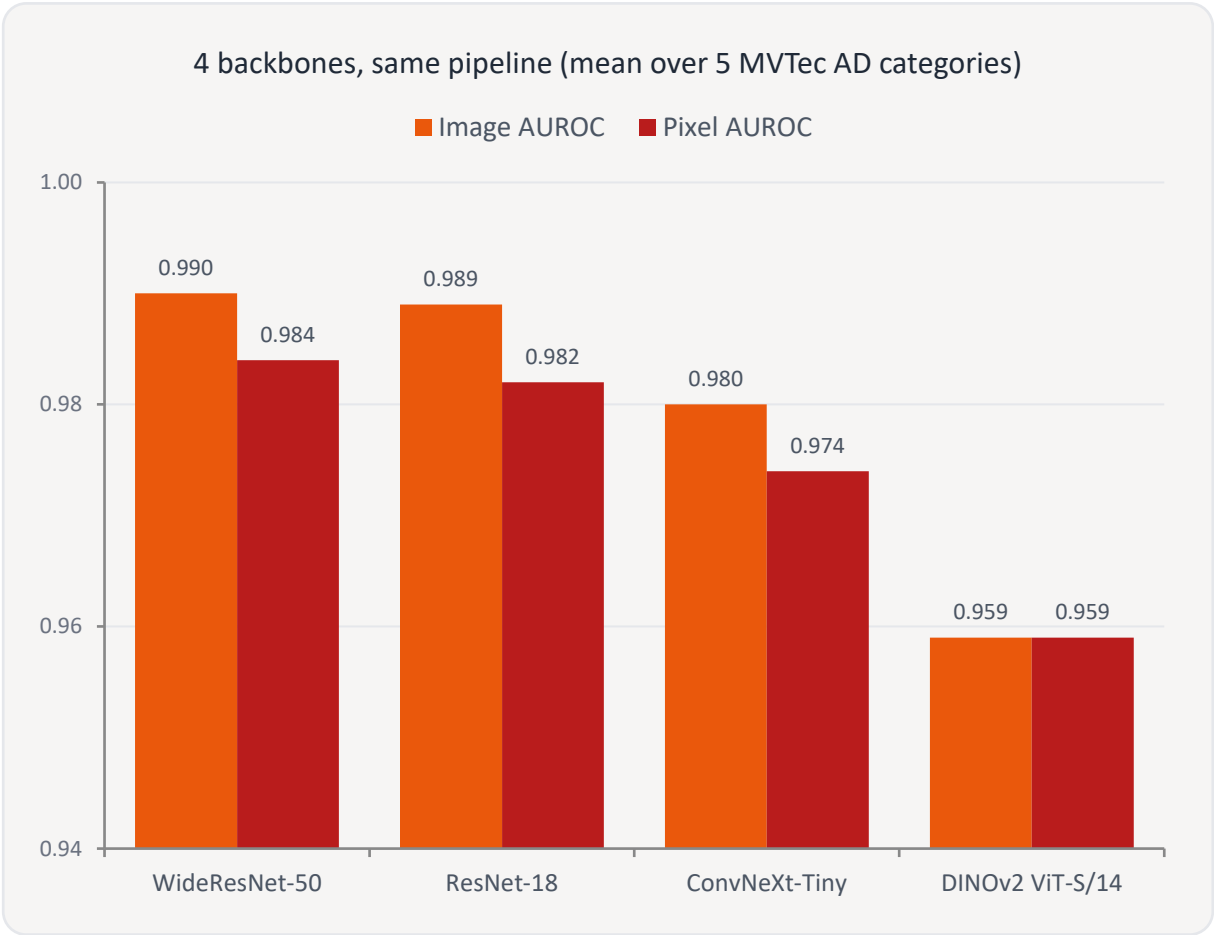
Try the live demo

priyanshu-singh-git.github.io/defectscope/

Code and full benchmark

github.com/Priyanshu-Singh-git/defectscope

The numbers behind it.



Versus published reference implementations

Method	Image	Pixel	Screw
DefectScope WideResNet-50 (mine)	0.990	0.984	0.969
DefectScope ResNet-18 (mine)	0.989	0.982	0.963
Intel anomalib PatchCore WideResNet-50	0.988	0.986	0.960
Intel anomalib PatchCore ResNet-18	0.980	0.984	0.943
Intel anomalib PaDiM WideResNet-50	0.961	0.984	0.845

Same 5 categories, image/pixel AUROC. Source: github.com/open-edge-platform/anomalib (v1.2.0 model benchmarks).
Keeping 1% of the memory bank (4 MB instead of 40 MB) costs 0.004 image AUROC.